MMDU MULLANA OFFICIAL SYLLABUS • BELE-001 • UNIT 4

Unit 4: Number Systems & Boolean Logic (How Computers Think)

Subject: Basic Electrical & Electronics Engineering (BELE-001) • **Exam Weightage:** 20 - 24 Marks • **Study Time:** 35 mins

Counting in Decimal, Binary, Octal, and Hexadecimal; 2's complement subtraction with zero tears; BCD addition with the famous +6 correction; Excess-3 self-complementing magic; Boolean algebra laws; De Morgan's theorems proven with 4 NAND gates.

1. Number Systems & 2's Complement Subtraction

1. Positional Number Systems & Inter-Radix Conversions

A positional number system represents numeric values using radix r (base) and a set of r discrete symbols $0, 1, ..., r-1$:

$$N = \sum_{i=-m}^{n-1} d_i \cdot r^i = d_{n-1}r^{n-1} + ... + d_1 r^1 + d_0 r^0 + d_{-1}r^{-1} + ... + d_{-m}r^{-m}$$

Common Digital Number Radices:

1. **Decimal ($r = 10$):** Human standard: $0, 1, 2, 3, 4, 5, 6, 7, 8, 9$.
2. **Binary ($r = 2$):** Digital logic standard: $0, 1$.
3. **Octal ($r = 8 = 2^3$):** Grouping of 3 binary bits: $0, 1, 2, 3, 4, 5, 6, 7$.
4. **Hexadecimal ($r = 16 = 2^4$):** Grouping of 4 binary bits: $0, 1, 2, 3, 4, 5, 6, 7, 8, 9$ A(10) B(11) C(12) D(13) E(14) F(15).

2. Signed Number Representations & 2's Complement Arithmetic

Computers represent signed integers using fixed-width registers (n bits):

- **Sign-Magnitude:** MSB is sign ($0 = +, 1 = -$), remaining $(n-1)$ bits represent magnitude. Has dual representation of zero ($+0$ and -0).
- **1's Complement:** Negative number obtained by inverting every bit: $[-A]_{1's} = (2^n - 1) - A$.
- **2's Complement (The Modern Processor Gold Standard):**

> $$[-\mathbf{A}]_{\mathbf{2's}} = \mathbf{2^n} - \mathbf{A} = [-\mathbf{A}]_{\mathbf{1's}} + \mathbf{1}$$

- Unique representation of zero (00000000_2).
- Asymmetric range for n bits: $[-\mathbf{2^{n-1}}, ; +\mathbf{2^{n-1}} - \mathbf{1}]$.

---

3. Rigorous Rules of 2's Complement Subtraction ($A - B$)

Compute $A + [-B]_{2's}$:

1. **Case 1:** $A \geq B$ **(Positive Result):**

- Addition yields an **End Carry of 1**.
- **Rule:** Discard the end carry. The remaining n-bit sum is strictly positive and directly represents the correct magnitude!

2. **Case 2:** $A < B$ **(Negative Result):**

- Addition yields **NO End Carry (0)**.
- **Rule:** The result is strictly negative and stored in 2's complement form. To recover the true decimal magnitude, take the 2's complement of the sum and attach a minus sign!

Problem:
Convert decimal number $(53)_{10}$ into: (1) Binary, (2) Hexadecimal.

Step 1 🚀: Repeated Division by 2 for Binary

- $53 \div 2 = 26$, remainder 1
- $26 \div 2 = 13$, remainder 0
- $13 \div 2 = 6$, remainder 1
- $6 \div 2 = 3$, remainder 0
- $3 \div 2 = 1$, remainder 1
- $1 \div 2 = 0$, remainder 1

Read remainders bottom-to-top: $\mathbf{(53)_{10} = (110101)_2}$.

Step 2 ⚡: Convert to Hexadecimal (Group in 4s)

> $$(0011; 0101)_2 = (3; 5)_{16} = \mathbf{(35)_{16}}$$

Check: $3 \times 16 + 5 = 48 + 5 = 53$.

🎯 $(53)_{10} = \mathbf{(110101)_2} = \mathbf{(35)_{16}}$.

University Question (6 Marks):
Perform the subtraction (28)_(10) - (13)_(10) using 8-bit 2's complement arithmetic.

Step 1 🚀: Convert Numbers to 8-Bit Binary

- +28 = 00011100_2

- +13 = 00001101_2

Step 2 ⚡: Find 2's Complement of 13 ([-13]_{2's})

- 1's complement of 13: 11110010_2

- Add 1: 11110010 + 1 = 11110011_2

Step 3 🎯: Add +28 and [-13]_{2's}
\begin{array}{r@{ }l}
00011100 & (+28) \\\n+ 11110011 & (-13) \\\n\hline
(1), 00001111 & (Sum)
\end{array}

Step 4 🏁: Handle End Carry
End carry of 1 is generated ⇒ **Discard end carry!**
Result is positive: 00001111_2 = 8 + 4 + 2 + 1 = $+15_{10}$. Matches 28 - 13 = 15!

🎯 Result: $00001111_2 = +\mathbf{15_{10}}$. End carry discarded.

University Question (8 Marks):
Perform the subtraction (14)_(10) - (39)_(10) using 8-bit 2's complement arithmetic. Explain how the negative result is recognized and evaluated.

Step 1 🚀: 8-bit Binary Representations

- +14 = 00001110_2

- +39 = 00100111_2

Step 2 ⚡: 2's Complement of Subtrahend +39

- 1's complement: 11011000

- 2's complement: 11011000 + 1 = 11011001_2

Step 3 🎯: Add 14 + [-39]_{2's}

$$\begin{array}{r@{ }l}
00001110 & (+14) \\\n+ 11011001 & (-39) \\\n\hline
11100111 & (No End Carry!)
\end{array}$$

Step 4 🏁: Interpret Result without End Carry

Since there is **NO end carry**, the result is **strictly negative** and stored in 2's complement form.

To find magnitude, take 2's complement of the sum:

- 1's complement of 11100111: 00011000
- Add 1: 00011000 + 1 = 00011001_2 = 16 + 8 + 1 = 25

Attach negative sign: -25_{10}. Matches 14 - 39 = -25!

🎯 No end carry ⇒ Result is Negative: $-(\mathbf{00011001})_{\mathbf{2}} = -\mathbf{25}_{\mathbf{10}}$.

2. BCD Code (+6 Rule) & Excess-3 Self-Complementing Magic

1. Binary Coded Decimal (BCD) & The +6 Correction Rule

8421 BCD (Natural BCD) Encoding:

In 8421 BCD, each decimal digit (0 through 9) is encoded as an independent 4-bit nibble using weights $8, 4, 2, 1$:

$$0_{10} = 0000_{\text{BCD}}, ...9_{10} = 1001_{\text{BCD}}$$

- A 4-bit register has $2^4 = 16$ possible binary states (0000_2 to 1111_2).
- The 6 code words from 1010_2 **(10_{10}) through** 1111_2 **(15_{10}) are ILLEGAL / FORBIDDEN states**!

---

2. Analytical Reason for the +6 Correction in BCD Addition

When two BCD digits are added using binary adders:

1. **Condition for Correction:** If the 4-bit binary sum exceeds 9 ($> 1001_2$) OR generates an auxiliary carry out of the nibble ($C_{\text{out}} = 1$), the sum has stepped into or through the forbidden zone!
2. **The +6 Rule:** The difference between hexadecimal radix (16) and decimal radix (10) is $16 - 10 = 6$. Adding binary 0110_2 (+6) skips directly past the 6 invalid states, forcing the correct decimal carry into the next higher nibble:

$$\text{BCD Sum} = (\text{Binary Sum}) + \mathbf{0110_2} (\text{if sum } > \mathbf{9} \text{ or carry } = \mathbf{1})$$

---

3. Excess-3 (XS-3) Code & Self-Complementing Property

Excess-3 is an unweighted code obtained by adding binary 0011_2 (+3) to each BCD digit:

$$\text{XS-3} = \text{BCD} + \mathbf{0011_2}$$

The Self-Complementing Theorem:

✅ **FUNDAMENTAL LAW / RULE:**
Theorem: If an Excess-3 code word is bitwise inverted (taking 1's complement), the resulting code word is identical to the Excess-3 representation of the **9's complement** of the original decimal digit!

$$\overline{\text{XS-3}(\mathbf{d})} = \text{XS-3}(\mathbf{9} - \mathbf{d})$$

- **Engineering Advantage:** Allows digital arithmetic units to perform decimal subtraction using standard adders and simple inverter gates without complex borrow-propagation circuitry!

Problem:
Convert the decimal number $(47)_{10}$ into: (1) 8421 BCD code, (2) Excess-3 code.

Step 1 🚀: 8421 BCD (Convert each digit individually into 4 bits)

- $4 = 0100_2$
- $7 = 0111_2$

$$(47)_{10} = \mathbf{0100}; \mathbf{0111}_{\text{BCD}}$$

Step 2 ⚡: Excess-3 Code (Add 3 to each digit)

- For digit 4: $4 + 3 = 7 = 0111_2$

- For digit 7: $7 + 3 = 10 = 1010_2$

> $$(47)_{10} = \mathbf{0111}; \mathbf{1010}_{\text{XS-3}}$$

🎯 $(47)_{10} = \mathbf{0100}; \mathbf{0111}_{\text{BCD}} = \mathbf{0111}; \mathbf{1010}_{\text{XS-3}}$.

University Question (6 Marks):
Perform the addition (38)_{BCD} + (45)_{BCD} using BCD arithmetic.

Step 1 🚀: Binary Addition of Nibbles

- 38 = 0011 ; 1000

- 45 = 0100 ; 0101

\begin{array}{r@{ }l}
0011 & 1000 \\\n+ 0100 & 0101 \\\n\hline
0111 & 1101 (7 and 13)
\end{array}

Step 2 ⚡: Check for Illegal Nibbles (> 9)

- Lower nibble is 1101_2 = 13 > 9 ⇒ **INVALID BCD!**

- Higher nibble is 0111_2 = 7 ≤ 9 ⇒ Valid.

Step 3 🎯: Add +6 (0110_2) to Lower Nibble
\begin{array}{r@{ }l}
0111 & 1101 \\\n+ 0000 & 0110 \\\n\hline
1000 & 0011 (8 and 3)
\end{array}
1000_2 = 8 and 0011_2 = 3 ⇒ (83)_(10). Matches 38 + 45 = 83!

🎯 BCD Sum: $\mathbf{1000}; \mathbf{0011}_{\text{BCD}} = (\mathbf{83})_{\mathbf{10}}$.

University Question (8 Marks):
Demonstrate the self-complementing property of Excess-3 code by finding the 9's complement of (62)_(10) using bitwise complementation in Excess-3.

Step 1 🚀: Represent (62)_(10) in Excess-3

- 6 + 3 = 9 = 1001_2

- 2 + 3 = 5 = 0101_2

(62)_(10) = 1001 ; 0101XS-3

Step 2 ⚡: Take Bitwise 1's Complement (Flip Every Bit)
\overline{1001 ; 0101} = 0110 ; 1010_2

Step 3 🎯: Decode Back to Decimal by Subtracting 3

- Higher nibble: 0110_2 = 6 ⇒ 6 - 3 = 3
- Lower nibble: 1010_2 = 10 ⇒ 10 - 3 = 7

Decoded number is 37_{10}.

Step 4 🏁: Mathematical Verification
The 9's complement of 62 is:
99 - 62 = 37_{10}
The bitwise NOT gate directly executed decimal 9's complement subtraction with zero arithmetic!

🎯 Proved: Excess-3 bitwise flip yields 0110; $1010_{\text{XS-3}} = \mathbf{37_{10}}$, exactly the 9's complement of 62.

3. Boolean Algebra & De Morgan's Theorems

1. Boolean Algebra Postulates & Fundamental Theorems

Boolean algebra is an algebraic system defined on the set $B = 0, 1$ with two binary operators: AND ($\cdot$), OR ($+$), and one unary operator: NOT ($'$ or $\overline{A}$).

Huntington's Postulates & Laws:

1. **Identity Laws:** $A + 0 = A, A \cdot 1 = A$
2. **Null / Dominance Laws:** $A + 1 = 1 A \cdot 0 = 0$
3. **Idempotent Laws:** $A + A = A, A \cdot A = A$
4. **Complementarity Laws:** $A + \overline{A} = 1 A \cdot \overline{A} = 0$
5. **Involution Law:** $\overline{\overline{A}} = A$
6. **Commutative Laws:** $A + B = B + A, A \cdot B = B \cdot A$
7. **Associative Laws:** $A + (B + C) = (A + B) + C, A(BC) = (AB)C$
8. **Distributive Laws:**

- $A \cdot (B + C) = AB + AC$
- $\mathbf{A + BC = (A + B)(A + C)}$ *(Dual distributive law — very powerful!)*

9. **Absorption Laws:** $A + AB = A, A(A + B) = A, A + \overline{A}B = A + B$

---

2. De Morgan's Dual Laws & Universal Logic Gates

✅ **FUNDAMENTAL LAW / RULE:**
First Theorem (NOR Gate Equivalence): The complement of a logical sum is equal to the logical product of the complements:

$$\mathbf{\overline{A + B} = \overline{A} \cdot \overline{B}}$$

✅ **FUNDAMENTAL LAW / RULE:**
Second Theorem (NAND Gate Equivalence): The complement of a logical product is equal to the logical sum of the complements:

$$\mathbf{\overline{A \cdot B} = \overline{A} + \overline{B}}$$

Universal Logic Gates:

- **NAND Gate and NOR Gate** are called **Universal Gates** because any conceivable Boolean function (AND, OR, NOT, XOR, XNOR) can be implemented using NAND gates alone or NOR gates alone.

- **NAND Implementation of Fundamental Gates:**

1. $\text{NOT}(A) = \overline{A \cdot A}$ (1 NAND gate)
2. $\text{AND}(A, B) = \overline{\overline{A \cdot B}}$ (2 NAND gates)
3. $\text{OR}(A, B) = \overline{\overline{A} \cdot \overline{B}}$ (3 NAND gates)
4. $\text{NOR}(A, B) = \overline{\overline{\overline{A} \cdot \overline{B}}}$ (4 NAND gates)
5. $\text{XOR}(A, B) = A\overline{B} + \overline{A}B$ (4 NAND gates)

Problem:
Simplify the Boolean expression $Y = AB + A\bar{B} + \bar{A}B$.

Step 1 🚀: Factor out common terms

> $$Y = A(B + \bar{B}) + \bar{A}B$$

Step 2 ⚡: Apply Complement Law ($B + \bar{B} = 1$)

> $$Y = A(1) + \bar{A}B = A + \bar{A}B$$

Step 3 🎯: Apply Distributive Law ($A + \bar{A}B = A + B$)

> $$Y = (A + \bar{A})(A + B) = (1)(A + B) = \mathbf{A + B}$$

🎯 Simplified: $\mathbf{Y = A + B}$ (Reduces from 3 AND gates to a single OR gate!).

University Question (6 Marks):
Verify De Morgan's Second Law: \overline{A · B} = \bar(A) + \bar(B) using a complete truth table.

Step 1 🚀: Truth Table Construction

A	B	A · B	\overline{A · B} (NAND)	\bar(A)	\bar(B)	\□ar{A + \bar(B)} (Bubbled OR)
0	0	0	**1**	1	1	**1**
0	1	0	**1**	1	0	**1**
1	0	0	**1**	0	1	**1**
1	1	1	**0**	0	0	**0**

Step 2 ⚡: Conclusion
Columns 4 and 7 are identically matching for all 4 possible input combinations!

🎯 Verified: $\overline{\mathbf{A \cdot B}} \equiv \bar{\mathbf{A}} + \bar{\mathbf{B}}$ (NAND gate is electrically equivalent to Bubbled OR).

University Question (10 Marks):
Show that a 2-input XOR gate (Y = A \bar(B) + \bar(A) B) can be synthesized using exactly four 2-input NAND gates with zero redundant gates. Provide algebraic proof for each gate output.

Step 1 🚀: Gate 1 (Central Cross NAND)
$G_1 = \overline{AB}$

Step 2 ⚡: Gates 2 and 3

- Gate 2 inputs are A and G_1:

$G_2 = \overline{A \cdot G_1} = \overline{A \cdot \overline{AB}} = \bar{A} + \overline{\overline{AB}} = \bar{A} + AB = (\bar{A} + A)(\bar{A} + B) = \bar{A} + B$

- Gate 3 inputs are B and G_1:

$G_3 = \overline{B \cdot G_1} = \overline{B \cdot \overline{AB}} = \bar{B} + AB = (\bar{B} + B)(\bar{B} + A) = A + \bar{B}$

Step 3 🎯: Gate 4 (Output Gate combining G_2 and G_3)
$Y = \overline{G_2 \cdot G_3} = \overline{(\bar{A} + B)(A + \bar{B})}$
Expand the inner product:
$(\bar{A} + B)(A + \bar{B}) = \bar{A} A + \bar{A} \bar{B} + BA + B \bar{B} = 0 + \bar{A} \bar{B} + AB + 0 = AB + \bar{A} \bar{B}$
Apply De Morgan's on the outer inversion:
$Y = \overline{AB + \bar{A}\bar{B}} = \overline{AB} \cdot \overline{\bar{A}\bar{B}} = (\bar{A} + \bar{B})(A + B) = \bar{A} B + A \bar{B} = A \oplus B$

🎯 Proved: Minimum 4 NAND gates synthesize $\mathbf{Y = A \oplus B = A\bar{B} + \bar{A}B}$.

University Solved Examination Questions (Unit 4)

MMDU Official University Solved Examination Questions (Unit 4)

Course: Basic Electrical & Electronics Engineering (`BELE-001`)
Pattern: Sections A (1 Mark Compulsory), B (2 Marks Analytical), C (6 Marks In-Depth), D (6 Marks Design/Long)

📘 Question 10 (1 Mark): Self-Complementing Property of Excess-3 Code

Question:
Demonstrate why Excess-3 code is termed a self-complementing code using decimal digit 4.

> 📊 **Question Schematic / State Specification:**

```
Question 9's Complement Inversion: 9's complement of decimal 4 is (9 - 4) = 5.
```

Step-by-Step Solution:

> ⚡ **KEY TAKEAWAY:**
> **Solution Analytical Derivation & Schematic:**

```
Solution Bitwise Inversion Trace:

        `Excess-3(4) = 4 + 3 = 7 = 0111_2`.

        `Bitwise NOT(0111_2) = 1000_2 = 8₁₀ = 5 + 3 = Excess-3(5)`!
```

Taking the 1's complement of an Excess-3 codeword directly produces the Excess-3 representation of its 9's complement, greatly simplifying digital decimal subtraction arithmetic.

🎯 **Verified Result:** Question 10 step-by-step resolution per MMDU university marking scheme.

📘 Question 11 (1 Mark): Why NAND is Termed a Universal Gate

Question:
Prove that a NOT gate can be synthesized using a single 2-input NAND gate.

> 📊 **Question Schematic / State Specification:**

```
Question Input Tie-Together: Connect both inputs of NAND together to input signal A.
```

Step-by-Step Solution:

> ⚡ **KEY TAKEAWAY:**
> **Solution Analytical Derivation & Schematic:**

```
Solution Boolean Idempotence Formulation:

        `Y = (A · A)' = A'` (By Boolean Idempotent Law).
```

By connecting both inputs of a 2-input NAND gate to common signal A, the output becomes `(A · A)' = A'`, functioning identically to an inverter (NOT gate).

🎯 **Verified Result:** Question 11 step-by-step resolution per MMDU university marking scheme.

📘 Question 12 (1 Mark): Gray Code Usage in Rotary Optical Encoders

Question:
Why is Gray code preferred over standard Binary code in optical shaft position encoders?

> 📊 **Question Schematic / State Specification:**

```
Question Binary Transition Glitch: Binary 7 (0111) → 8 (1000) flips 4 bits simultaneously; Gray flips only 1 bit.
```

Step-by-Step Solution:

> ⚡ **KEY TAKEAWAY:**
> **Solution Analytical Derivation & Schematic:**

```
Solution Unit Distance Property:

        `Hamming Distance = 1` between adjacent counts ⟹ eliminates intermediate spurious read codes.
```

In standard binary, multiple bits switch simultaneously (e.g. 0111 to 1000). Due to physical sensor misalignment, bits switch at slightly different nanoseconds, creating erroneous intermediate spikes. Gray code changes only one bit at a time, preventing read errors.

🎯 **Verified Result:** Question 12 step-by-step resolution per MMDU university marking scheme.

📘 Question 16 (2 Marks): 2's Complement Subtraction

Question:

Perform subtraction (28)_(10) - (15)_(10) using 8-bit 2's complement arithmetic.

> 📊 **Question Schematic / State Specification:**

```
Question Binary Operand Conversion: 28 = 00011100_2 | 15 = 00001111_2.
```

Step-by-Step Solution:

> ⚡ **KEY TAKEAWAY:**
> **Solution Analytical Derivation & Schematic:**

```
Solution 2's Complement Addition Trace:

        `1's Complement of 15: 11110000_2`

        `2's Complement of 15: 11110000 + 1 = 11110001_2`

        `Add to 28: 00011100 + 11110001 = 1 00001101_2`

        `Discard End Carry (1) ⟹ Result = 00001101_2 = 8 + 4 + 1 = (13)_(10) (Correct!)`
```

The end carry indicates a positive result, yielding `00001101_2 = 13₁₀`, matching 28 - 15 = 13.

🎯 **Verified Result:** Question 16 step-by-step resolution per MMDU university marking scheme.

📘 Question 20 (4 Marks): 4-Variable K-Map Minimization with Don't Cares

Question:
Minimize the Boolean function: F(A, B, C, D) = Σm(1, 3, 7, 11, 15) + d(0, 2, 5) using a 4-variable K-Map. Implement using basic gates.

> 📊 **Question Schematic / State Specification:**

```
Question K-Map Cell Mapping: Minterms at 1, 3, 7, 11, 15; Don't cares at 0, 2, 5.
```

Step-by-Step Solution:

> ⚡ **KEY TAKEAWAY:**
> **Solution Analytical Derivation & Schematic:**

```
Solution Optimal Grouping Strategy:

        `Group 1 (Octet): Combine minterms m₁, m₃, m₇, m₁1, m₁5 with don't cares d₀, d₂, d₅!`

        `Cells (0, 1, 2, 3, 5, 7, 11, 15): Notice column CD = (m₃, m₇, m₁1, m₁5) forms a Quad CD`

        `Row A'B' (0, 1, 2, 3) + (5, 7): Combine (1, 3, 5, 7) with (3, 7, 11, 15) ⟹`

        `Quad 1: (3, 7, 11, 15) yields CD`

        `Quad 2: (1, 3, 5, 7) yields A'D`

        `Quad 3: (0, 1, 2, 3) with d₀, d₂ yields A'B'`

        `Minimal SOP Expression: F = CD + A'B' + A'D (or F = CD + A'B')`
```

By leveraging Don't Care cells d(0, 2), the function simplifies down to `F = CD + A'B'`, eliminating 3 gates.

🎯 **Verified Result:** Question 20 step-by-step resolution per MMDU university marking scheme.